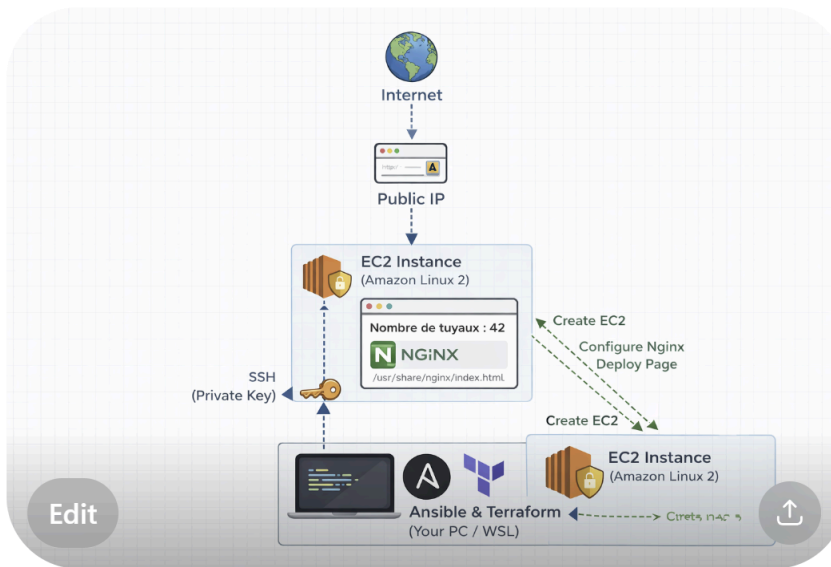


Table of Contents

1 LAB COMPLET.....	2
2 Objectif.....	2
Objectif du lab	2
Résultat attendu	2
3 Prérequis.....	2
3.1 ansible et Terraform.....	2
3.2 AWS configuré :.....	3
3.3 Clé SSH :.....	3
3.3.1 Méthode 1 (RECOMMANDÉE) — via AWS.....	3
3.3.2 Autre Option : clé dans le projet (lab uniquement).....	4
Structure :	4
3.3.3 Méthode 2 (optionnelle) — générer soi-même la clef.....	5
4 Structure du projet.....	6
5 Terraform.....	6
6 Lancer Terraform.....	8
7 Ansible.....	10
7.1 inventory.ini.....	10
7.2 ansible.cfg.....	11
7.3 Playbook.....	11
7.4 Rôle nginx.....	11
7.5 fichier ansible.cfg.....	11
7.6 fichier index.....	12
7.7 handler.....	12
7.8 structure finale.....	12
8 Exécuter Ansible.....	13
9 Test.....	14
10 terraform destroy.....	14



1 LAB COMPLET

WSL + Terraform + AWS + Ansible + rôle nginx

2 Objectif

Objectif du lab

Ce lab a pour objectif de mettre en place une chaîne complète d'automatisation **Infrastructure + Configuration** en utilisant **Terraform, AWS et Ansible**.

L'objectif est de comprendre comment :

- Provisionner une infrastructure cloud sur **AWS** avec Terraform
 - Configurer automatiquement les serveurs avec Ansible
 - Appliquer les bonnes pratiques Ansible (roles, variables, templates, handlers)
 - Créer un déploiement reproductible, modulaire et maintenable
-

Résultat attendu

À la fin du lab, une instance EC2 sera :

- créée automatiquement avec Terraform
- accessible en SSH via une clé privée
- configurée avec Ansible
- exposée via un serveur web (Nginx)

Une page web sera accessible et affichera :

Nombre de tuyaux : 42

3 Prérequis

3.1 ansible et Terraform

Dans WSL (/home/henri) :

Si terraform pas installé :
Installer le package terraform dans Ubuntu

```
henri@Henri:~$  
henri@Henri:~$ sudo apt-get update  
sudo apt-get install terraform
```

Ensuite , vérifier la version de terraform
terraform -v

```
henri@Henri:~$ terraform -v  
Terraform v1.14.8  
on linux_amd64  
henri@Henri:~$  
henri@Henri:~$
```

Ensuite , si ansible non installé, alors l'installer dans Ubuntu

Ensuite : vérifier la version de ansible
ansible --version

```
henri@Henri:~$  
henri@Henri:~$ ansible --version  
ansible [core 2.16.3]  
  config file = None  
  configured module search path = ['/home/henri/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']  
  ansible python module location = /usr/lib/python3/dist-packages/ansible  
  ansible collection location = /home/henri/.ansible/collections:/usr/share/ansible/collections  
  executable location = /usr/bin/ansible  
  python version = 3.12.3 (main, Mar  3 2026, 12:15:18) [GCC 13.3.0] (/usr/bin/python3)  
  jinja version = 3.1.2  
  libyaml = True  
henri@Henri:~$
```

3.2 AWS configuré :

aws configure

3.3 Clé SSH :

3.3.1 Méthode 1 (RECOMMANDÉE) — via AWS

👉 C'est la méthode la plus simple pour débiter

1 Créer une key pair dans AWS

Dans AWS :

Va dans EC2

Menu gauche → Key Pairs

Clique Create key pair

Paramètres :

Name: mykeypair

Type: RSA

Format: .pem

👉 Cliquez Create

2 Téléchargement

👉 AWS télécharge automatiquement :

mykeypair.pem

⚠ IMPORTANT :

tu ne peux le télécharger qu'une seule fois

3 Déplacer la clé

Dans ton terminal :

mkdir -p ~/.ssh

mv ~/Downloads/mykeypair.pem ~/.ssh/

4 Sécuriser la clé (obligatoire)

chmod 400 ~/.ssh/mykeypair.pem

👉 Sinon SSH refusera la connexion

Tester

```
ssh -i ~/.ssh/mykeypair.pem ec2-user@IP_PUBLIC
```



Lien avec Terraform

Dans ton `main.tf` :

```
key_name = " mykeypair "
```



⚠ le nom doit être EXACTEMENT le même que dans AWS



Lien avec Ansible

Inventory :

```
[web]
```

```
IP_PUBLIC ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/  
mykeypair.pem
```

3.3.2 Autre Option : clé dans le projet (lab uniquement)

Structure :

```
project/  
├── terraform/  
├── ansible/  
├── keys/  
│   └── mykeypair
```

Permissions

```
chmod 400 keys/mykeypair
```



Ansible

```
ansible_ssh_private_key_file=../keys/mykeypair
```



SSH

```
ssh -i ../keys/mykeypair ec2-user@IP
```

3.3.3 Méthode 2 (optionnelle) — générer soi-même la clef



plus avancé (utile plus tard)

```
ssh-keygen -t rsa -b 2048 -f ~/.ssh/my-keypair
```



ça crée :

```
~/.ssh/my-keypair
```

```
~/.ssh/my-keypair.pub
```

Mais dans AWS il faut importer la clé → plus complexe



Vérifier

```
ls ~/.ssh/my-keypair*
```



Permissions

```
chmod 400 ~/.ssh/my-keypair
```



2. Importer dans AWS

Étapes :

Va dans EC2

Menu gauche → Key Pairs

Clique Import key pair

Paramètres :

Name: my-keypair ⚠ (important pour Terraform)

Public key content :

👉 copie le contenu de :

```
cat ~/.ssh/my-keypair.pub
```

👉 ça ressemble à :

```
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQ... user@host
```



colle ça dans AWS



clique Import key pair



3. Utilisation avec Terraform

Dans ton main.tf :

```
key_name = "my-keypair"
```

👉 doit matcher EXACTEMENT



4. Utilisation avec Ansible

[web]

```
IP_PUBLIC ansible_user=ec2-user ansible_ssh_private_key_file=~/.ssh/my-keypair
```

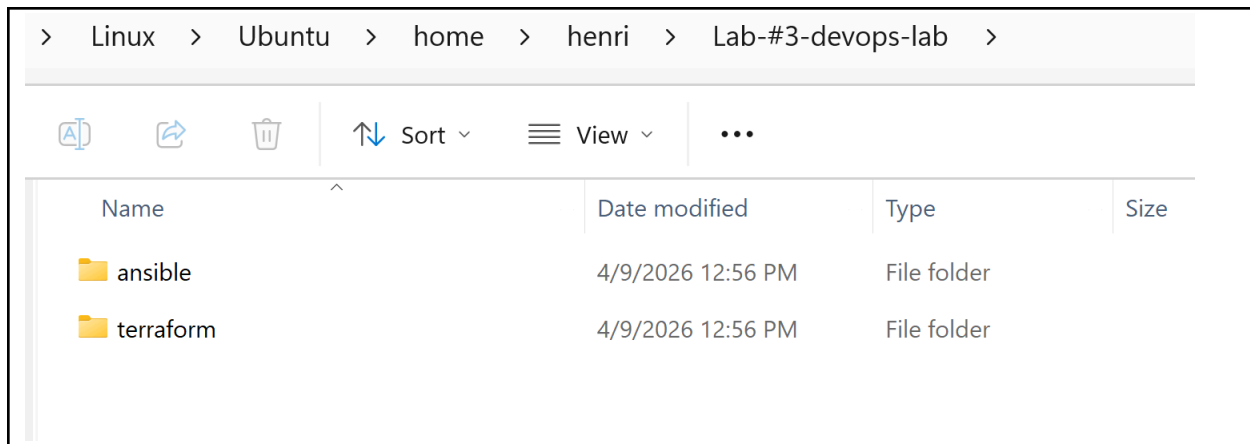
4 Structure du projet

```
cd ~
```

```
mkdir -p
```

```
Lab-#3-devops-lab/{terraform,ansible/playbooks,ansible/roles/nginx/tasks}
```

```
cd Lab-#3-devops-lab
```



5 Terraform

👉 terraform/main.tf

```
provider "aws" {
  region = "ca-central-1"
}

# 🔍 AMI dynamique Amazon Linux 2
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}

resource "aws_security_group" "web_sg" {
  name = "web-sg"

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # pour test
  }

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
```

```

        from_port    = 0
        to_port      = 0
        protocol      = "-1"
        cidr_blocks   = ["0.0.0.0/0"]
    }
}

resource "aws_instance" "web" {
    ami            = data.aws_ami.amazon_linux.id
    instance_type  = "t3.micro"
    key_name       = var.key_name

    vpc_security_group_ids = [aws_security_group.web_sg.id]

    tags = {
        Name = "ansible-web"
    }
}

output "public_ip" {
    value = aws_instance.web.public_ip
}

```

👉 terraform/variables.tf

```

variable "key_name" {
    description = "AWS key pair name"
}

```

Créer terraform.tfvars

→ key_name = "mykeypair"

6 Lancer Terraform

```

cd Lab-#3-devops-lab
terraform plan
terraform init
terraform apply

```

👉 note l'IP affichée

Terraform init :

```
henri@Henri:~/Lab-#3-devops-lab/terraform$  
henri@Henri:~/Lab-#3-devops-lab/terraform$ terraform init  
Initializing the backend...  
Initializing provider plugins...  
- Finding latest version of hashicorp/aws...  
- Installing hashicorp/aws v6.40.0...  
- Installed hashicorp/aws v6.40.0 (signed by HashiCorp)  
Terraform has created a lock file .terraform.lock.hcl to record the provider  
selections it made above. Include this file in your version control repository  
so that Terraform can guarantee to make the same selections by default when  
you run "terraform init" in the future.  
  
Terraform has been successfully initialized!  
  
You may now begin working with Terraform. Try running "terraform plan" to see  
any changes that are required for your infrastructure. All Terraform commands  
should now work.  
  
If you ever set or change modules or backend configuration for Terraform,  
rerun this command to reinitialize your working directory. If you forget, other  
commands will detect it and remind you to do so if necessary.  
henri@Henri:~/Lab-#3-devops-lab/terraform$
```

Terraform plan

```
+ name           = "web-sg"  
+ name_prefix    = (known after apply)  
+ owner_id       = (known after apply)  
+ region         = "ca-central-1"  
+ revoke_rules_on_delete = false  
+ tags_all       = (known after apply)  
+ vpc_id         = (known after apply)  
}  
  
Plan: 2 to add, 0 to change, 0 to destroy.  
  
Changes to Outputs:  
+ public_ip = (known after apply)  
  
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if  
you run "terraform apply" now.  
henri@Henri:~/Lab-#3-devops-lab/terraform$
```

Terraform apply

```
aws_security_group.web_sg: Creating...  
aws_security_group.web_sg: Creation complete after 3s [id=sg-0661e7821eb71d757]  
aws_instance.web: Creating...  
aws_instance.web: Still creating... [00m10s elapsed]  
aws_instance.web: Creation complete after 13s [id=i-01eae4c909833992d]  
  
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.  
  
Outputs:  
  
public_ip = "15.222.239.179"  
henri@Henri:~/Lab-#3-devops-lab/terraform$
```

Terraform state list

```
henri@Henri:~/Lab-#3-devops-lab/terraform$ terraform state list
data.aws_ami.amazon_linux
aws_instance.web
aws_security_group.web_sg
henri@Henri:~/Lab-#3-devops-lab/terraform$
```

Test SSH :

Test ssh afin de vérifier si on peut joindre l'instance EC2 qui a été crée dans l'étape précédente par terraform

ssh -i ~/.ssh/mykeypair.pem ec2-user@15.222.239.179

```
henri@Henri:~/Lab-#3-devops-lab/terraform$ ssh -i ~/.ssh/mykeypair.pem ec2-user@15.222.239.179
#_
~\#####_      Amazon Linux 2
~~\#####\
~~\###|
~~\#/
~~\V~'---
~~~
~~~.---
~~~/_/_/_/_/_
~/m/'

AL2 End of Life is 2026-06-30.

A newer version of Amazon Linux is available!

Amazon Linux 2023, GA and supported until 2028-03-15.
https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-172-31-12-69 ~]$ exit
Logout
Connection to 15.222.239.179 closed.
henri@Henri:~/Lab-#3-devops-lab/terraform$
```

Test dans AWSconsole

Dans AWS, on vérifie la présence de l'instance EC2 qui a été crée



The screenshot shows the AWS Management Console interface for EC2 Instances. The left sidebar contains navigation links: EC2, Dashboard, AWS Global View, Events, Instances (selected), Instance Types, and Launch Templates. The main panel displays a table of instances with columns: Name, Instance ID, Instance state, Instance type, Status check, Alarm status, and Availability Zone. Two instances are listed: 'test' (Stopped) and 'ansible-web' (Running). The 'ansible-web' instance is highlighted with a red box, and its 'Instance type' 't3.micro' is also highlighted with a red box. The 'Status check' for 'ansible-web' shows '3/3 checks passed'.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
test	i-0ff9031b45ab97572	Stopped	t3.micro	-	View alarms +	ca-central-1b
ansible-web	i-01ee4c909833992d	Running	t3.micro	3/3 checks passed	View alarms +	ca-central-1b

7 Ansible

```
cd Lab-#3-devops-lab/ansible
```

7.1 inventory.ini

```
[web]
ec2 ansible_host=IP ansible_user=ec2-user
ansible_ssh_private_key_file=~/.ssh/mykeypair.pem
```

👉 **replace** IP par : 15.222.239.179

7.2 ansible.cfg

```
echo -e "[defaults]\nhost_key_checking = False\ninventory = inventory.ini" >
ansible.cfg
```

7.3 Playbook

👉 `playbooks/site.yml`

```
- hosts: web
  become: yes
  roles:
    - nginx
```

7.4 Rôle nginx

👉 `roles/nginx/tasks/main.yml`

```
- name: Enable nginx repo
  command: amazon-linux-extras enable nginx1

- name: Install nginx
  yum:
    name: nginx
    state: present

- name: Deploy web page
  template:
```

```
    src: index.html.j2
    dest: /usr/share/nginx/html/index.html
    notify: restart nginx
```

```
- name: Start nginx
  service:
    name: nginx
    state: started
    enabled: true
```

7.5 fichier ansible.cfg

Créer un fichier ansible.cfg dans le répertoire ansible

Contenu du fichier :

```
[defaults]
roles_path = ./roles
inventory = ./inventory.ini
```

7.6 fichier index

Créer le répertoire templates et le fichier index

👉 roles/nginx/templates/index.html.j2

Contenu du fichier index.html.j2 :

```
<h1>Nombre de tuyaux : 42</h1>
```

7.7 handler

👉 roles/nginx/handlers/main.yml

Contenu du fichier main.yml qui est dans le répertoire handler :

```
- name: restart nginx
  service:
    name: nginx
    state: restarted
```

7.8 structure finale

Linux > Ubuntu > home > henri > Lab-#3-devops-lab > ansible >				
Name	Date modified	Type	Size	
playbooks	4/9/2026 6:00 PM	File folder		
roles	4/9/2026 12:56 PM	File folder		
inventory.ini	4/9/2026 5:49 PM	Configuration sett...	1 KB	
ansible.cfg	4/9/2026 6:17 PM	CFG File	0 KB	

```
ansible/
├─ ansible.cfg
├─ inventory.ini
├─ playbooks/
│   └─ site.yml
├─ roles/
│   └─ nginx/
│       ├── tasks/
│       │   └─ main.yml
│       ├── templates/
│       │   └─ index.html.j2
│       └─ handlers/
│           └─ main.yml
```

8 Exécuter Ansible

```
ansible-playbook playbooks/site.yml -i inventory.ini
```

ou bien :

```
ansible-playbook playbooks/site.yml
```

```

henri@Henri:~/Lab-#3-devops-lab/ansible$
henri@Henri:~/Lab-#3-devops-lab/ansible$ ansible-playbook playbooks/site.yml

PLAY [web] *****

TASK [Gathering Facts] *****
[WARNING]: Platform linux on host ec2 is using the discovered Python interpreter at /usr/bin/python3.7, but future
installation of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-
core/2.16/reference_appendices/interpreter_discovery.html for more information.
ok: [ec2]

TASK [nginx : Enable nginx repo] *****
changed: [ec2]

TASK [nginx : Install nginx] *****
ok: [ec2]

TASK [nginx : Deploy web page] *****
changed: [ec2]

TASK [nginx : Start nginx] *****
changed: [ec2]

RUNNING HANDLER [nginx : restart nginx] *****
changed: [ec2]

PLAY RECAP *****
ec2                : ok=6    changed=4    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

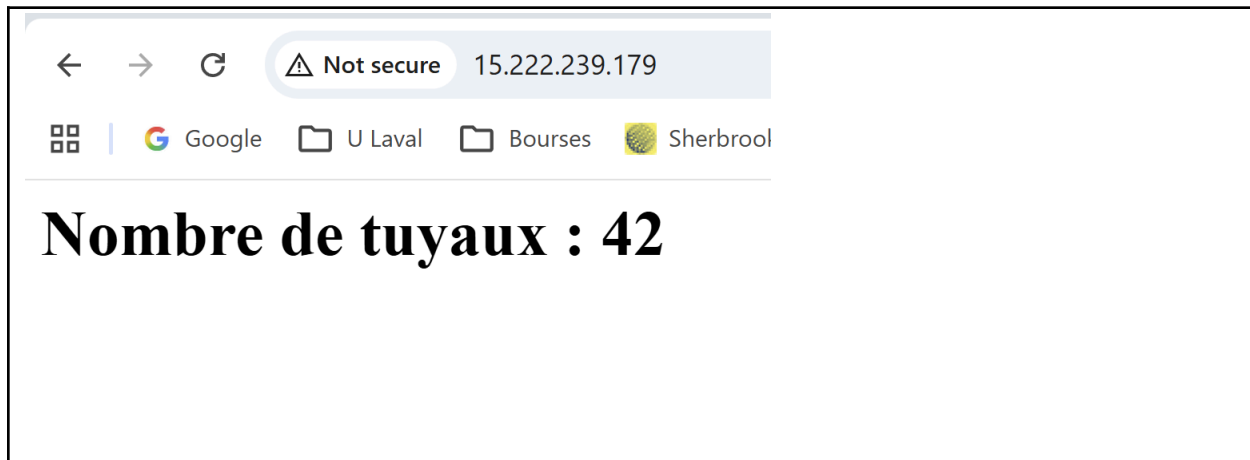
henri@Henri:~/Lab-#3-devops-lab/ansible$

```

9 Test

👉 navigateur :

<http://15.222.239.179>



10 terraform destroy

```
cd ~/Lab-#3-devops-lab/terraform
terraform destroy
```

```
Plan: 0 to add, 0 to change, 2 to destroy.
```

```
Changes to Outputs:
```

```
- public_ip = "15.222.239.179" -> null
```

```
Do you really want to destroy all resources?
```

```
Terraform will destroy all your managed infrastructure, as shown above.
```

```
There is no undo. Only 'yes' will be accepted to confirm.
```

```
Enter a value: yes
```

```
aws_instance.web: Destroying... [id=i-01eae4c909833992d]
```

```
aws_instance.web: Still destroying... [id=i-01eae4c909833992d, 00m08s elapsed]
```

```
aws_instance.web: Still destroying... [id=i-01eae4c909833992d, 00m18s elapsed]
```

```
aws_instance.web: Still destroying... [id=i-01eae4c909833992d, 00m28s elapsed]
```

```
aws_instance.web: Destruction complete after 28s
```

```
aws_security_group.web_sg: Destroying... [id=sg-0661e7821eb71d757]
```

```
aws_security_group.web_sg: Destruction complete after 1s
```

```
Destroy complete! Resources: 2 destroyed.
```

```
henri@Henri:~/Lab-#3-devops-lab/terraform$
```

```
henri@Henri:~/Lab-#3-devops-lab/terraform$
```

```
henri@Henri:~/Lab-#3-devops-lab/terraform$
```